

CineMax

Manuale Tecnico

Laboratorio Interdisciplinare B — Sviluppo GUI e architettura Client/Server

Renee Angelica Cabigting (756997, CO)

Indice

Indice.....	2
1. Introduzione e architettura generale.....	3
2. Progettazione del software — diagramma delle classi.....	3
3. Design pattern utilizzati	4
3.1 Data Access Object (DAO)	4
3.2 Command (protocollo di rete)	4
3.3 Data Transfer Object (DTO).....	4
3.4 Factory (connessioni al database).....	4
3.5 Separazione in livelli (layered architecture)	4
4. Progettazione del database	4
4.1 Scelte di ristrutturazione principali.....	4
4.2 Vincoli non esprimibili con semplici CHECK.....	5
5. Gestione della concorrenza	5
5.1 Livello di rete (server)	5
5.2 Livello di dati (database)	5
6. Gestione degli errori e delle eccezioni.....	6
7. Protocollo di comunicazione e strutture dati	6

1. Introduzione e architettura generale

CineMax (CM) è una piattaforma per la gestione di un cinema monosala, sviluppata secondo un'architettura client/server a tre livelli logici: un database relazionale PostgreSQL (dbCM), un modulo serverCM che espone i servizi applicativi tramite un protocollo proprietario su socket TCP, e un modulo clientCM con interfaccia grafica Swing.

I tre moduli sono organizzati come progetto Maven multi-modulo:

- cinemax-common — protocollo di comunicazione (Comando, Richiesta, Risposta) e DTO condivisi tra client e server;
- cinemax-server — accesso al database (DAO), logica applicativa (servizi) e livello di rete (ServerCM, GestoreClient, Dispatcher);
- cinemax-client — livello di comunicazione (ConnessioneServer) e interfaccia grafica Swing (FinestraPrincipale e le schermate).

Questa separazione in moduli riflette la separazione richiesta dalla traccia tra serverCM e clientCM, permettendo di generare due eseguibili .jar indipendenti a partire da una base di codice condivisa (cinemax-common), evitando duplicazione delle classi che rappresentano i dati scambiati sulla rete.

2. Progettazione del software — diagramma delle classi

Il diagramma seguente mostra le classi principali dei tre moduli e le relazioni tra loro. Le classi sono raggruppate per modulo (cluster tratteggiati) per evidenziare i confini architetturali: nessuna classe di cinemax-client dipende da classi di cinemax-server, e viceversa — l'unico punto di contatto è cinemax-common.

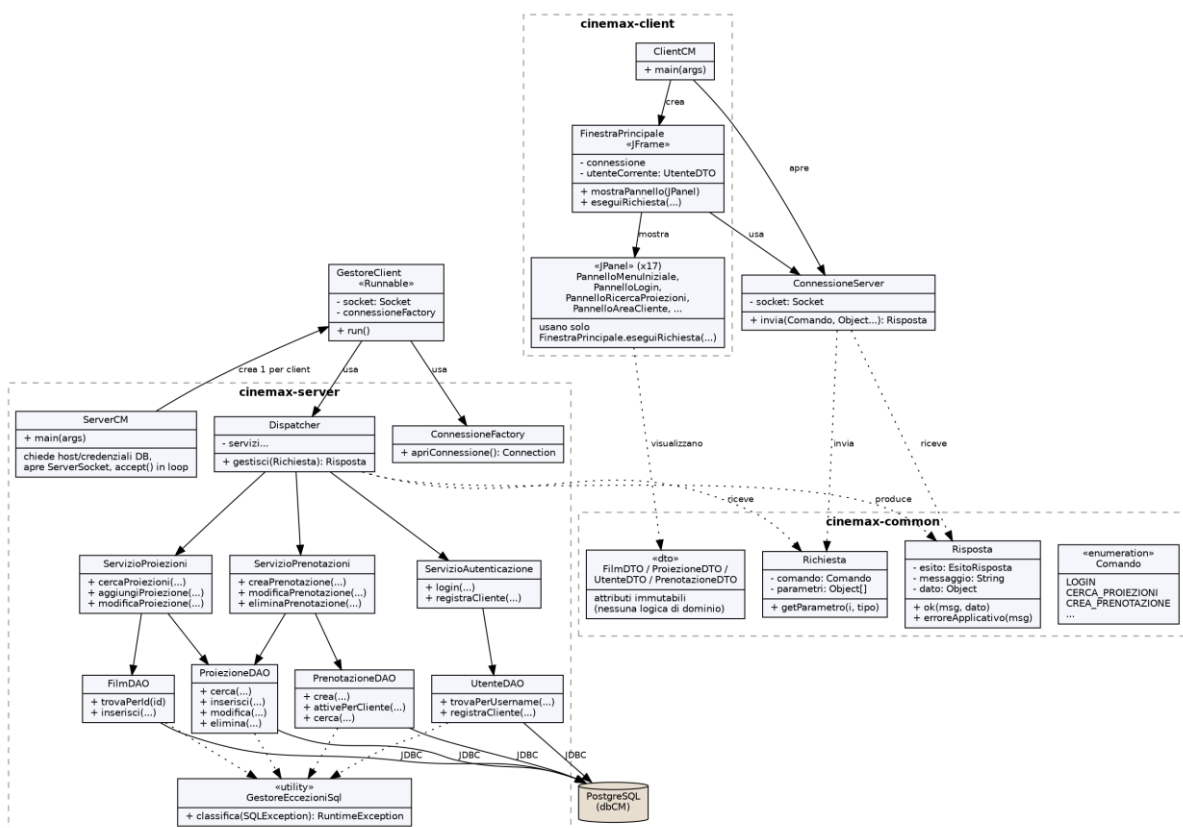


Figura 1 — Diagramma delle classi principali di CineMax

3. Design pattern utilizzati

3.1 Data Access Object (DAO)

Ogni entità del database (Film, Proiezione, Utente, Prenotazione) ha un DAO dedicato (FilmDAO, ProiezioneDAO, UtenteDAO, PrenotazioneDAO) che incapsula tutto l'accesso JDBC. I livelli superiori (servizi) non contengono mai SQL diretto: questo isola l'eventuale futura sostituzione del DBMS o dell'ORM a un solo livello del codice.

3.2 Command (protocollo di rete)

Ogni operazione richiedibile dal client è rappresentata da un valore dell'enumerazione Comando, incapsulato in un oggetto Richiesta generico (comando + parametri posizionali). Il Dispatcher lato server funge da invoker che smista ogni comando verso il metodo di servizio corrispondente. Questo evita di dover definire una classe concreta per ciascuna delle 17 operazioni del protocollo.

3.3 Data Transfer Object (DTO)

FilmDTO, ProiezioneDTO, UtenteDTO e PrenotazioneDTO sono oggetti immutabili, privi di logica di dominio, usati esclusivamente per il trasferimento di dati sulla rete (tramite serializzazione Java). Sono volutamente distinti dalle eventuali entità di persistenza lato server, cosa che disaccoppia il protocollo di rete dai dettagli implementativi dello strato di accesso ai dati.

3.4 Factory (connessioni al database)

ConnessioneFactory incapsula la creazione delle connessioni JDBC (host, porta, credenziali) dietro un unico metodo `apriConnessione()`, usato da ogni `GestoreClient` per ottenere una connessione dedicata alla propria sessione.

3.5 Separazione in livelli (layered architecture)

Sia lato server (rete → servizi → DAO → database) sia lato client (GUI → comunicazione → protocollo), il codice è organizzato in livelli con dipendenze a senso unico: un livello superiore conosce quello inferiore, mai il contrario. Ad esempio, `ProiezioneDAO` non sa nulla del protocollo di rete, e nessuna schermata Swing importa mai `java.net.Socket`.

4. Progettazione del database

Il database `dbCM` è stato progettato con l'approccio classico a tre fasi (schema concettuale ER, ristrutturazione, traduzione in schema relazionale), verificando che ogni tabella soddisfi la Terza Forma Normale (3NF). La documentazione completa e dettagliata si trova nel file `db/progettazione-DB.md` allegato al repository; questa sezione ne riporta una sintesi.

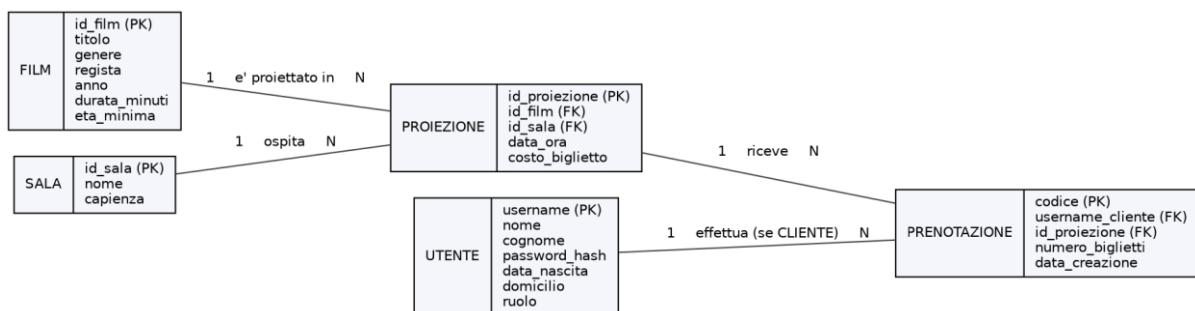


Figura 2 — Schema Entità-Relazione del database `dbCM`

4.1 Scelte di ristrutturazione principali

- Introduzione dell'entità Sala (non esplicitamente richiesta) per evitare di codificare la capienza come costante applicativa, e per rendere lo schema estendibile a un futuro cinema multisala;
- Nessun attributo derivato memorizzato: il numero di posti liberi di una proiezione non è una colonna della tabella, ma viene sempre calcolato con una query aggregata (capienza – somma biglietti prenotati), evitando anomalie di aggiornamento;
- Chiavi naturali (username, codice prenotazione) dove già uniche per requisito applicativo; chiavi surrogate (id_film, id_sala, id_proiezione) altrove.

4.2 Vincoli non esprimibili con semplici CHECK

Alcuni vincoli richiedono di esaminare più righe o più tabelle contemporaneamente, cosa che un CHECK constraint di PostgreSQL non può fare. Sono quindi implementati con funzioni trigger:

- trg_verifica_capienza — impedisce di prenotare più posti di quanti liberi ne restano per una proiezione;
- trg_verifica_sovrapposizione — impedisce due proiezioni sovrapposte nella stessa sala, considerando la durata del film;
- trg_verifica_ruolo_cliente — impedisce che un utente non CLIENTE risulti associato a una prenotazione;
- trg_blocca_update_proiezione / trg_blocca_delete_proiezione — impediscono di modificare o eliminare una proiezione che ha già prenotazioni;
- trg_genera_codice_prenotazione — genera automaticamente il codice progressivo (PRN-0001, ...) tramite una SEQUENCE, atomica per definizione in PostgreSQL.

5. Gestione della concorrenza

La traccia richiede che il sistema supporti più client connessi in parallelo, con possibili accessi concorrenti alle stesse risorse. La soluzione adottata opera su due livelli distinti:

5.1 Livello di rete (server)

ServerCM usa il modello "thread-per-connessione": ogni client accettato da `ServerSocket.accept()` viene gestito da un'istanza dedicata di `GestoreClient`, eseguita su un proprio Thread. Il server torna immediatamente in ascolto di nuove connessioni senza attendere che il client corrente termini. Ogni `GestoreClient` apre una propria `Connection JDBC`, mantenuta per l'intera sessione.

5.2 Livello di dati (database)

I thread Java possono eseguire operazioni in parallelo senza bloccarsi a vicenda, ma le operazioni che toccano dati condivisi (capienza di una proiezione, sovrapposizione tra proiezioni) devono essere serializzate a livello di database. I trigger critici usano `SELECT ... FOR UPDATE` per acquisire un lock di riga sulla proiezione coinvolta prima di leggere/validare i dati aggregati: se due client tentano di prenotare contemporaneamente l'ultimo posto disponibile, la seconda transazione attende il commit della prima e rivaluta la condizione con i dati aggiornati.

Il diagramma seguente illustra questo meccanismo nel caso d'uso di creazione di una prenotazione:

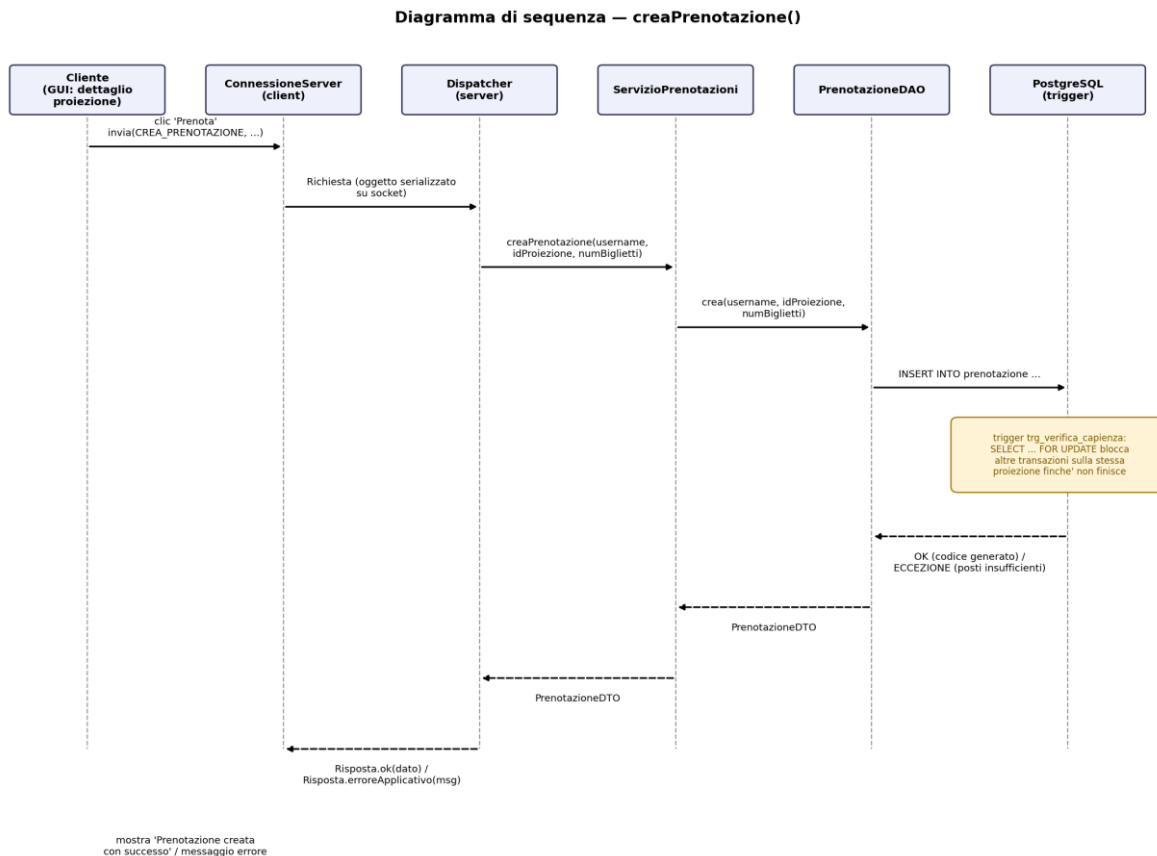


Figura 3 — Diagramma di sequenza: creazione di una prenotazione

6. Gestione degli errori e delle eccezioni

Le eccezioni sono classificate in due categorie distinte lungo tutto il livello server:

- **EccezioneApplicativa** — violazione di una regola di business prevista (es. posti insufficienti, credenziali errate, username duplicato). Il messaggio è pensato per essere mostrato così com'è all'utente.
- **EccezioneTecnica** — problema tecnico non previsto (es. connessione al database persa). Il messaggio è pensato per il log, non per l'utente finale.

La classe `GestoreEccezioniSql` traduce ogni `SQLException` in una delle due eccezioni sopra, in base al suo `SQLState`: il codice `P0001` (`RAISE EXCEPTION` nei trigger PL/pgSQL) e `23505` (violazione di vincolo `UNIQUE`) diventano `EccezioneApplicativa`; qualunque altro stato diventa `EccezioneTecnica`.

Il `Dispatcher` cattura sempre queste eccezioni e le traduce in un oggetto `Risposta` con l'esito corrispondente (`OK`, `ERRORE_APPLICATIVO`, `ERRORE_TECNICO`): nessuna eccezione Java grezza (né tantomeno uno stack trace) viene mai trasmessa sul socket al client.

Lato client, `FinestraPrincipale.eseguiRichiesta(...)` esegue ogni chiamata al server su uno `SwingWorker` (per non bloccare l'interfaccia) e mostra un `JOptionPane` con il messaggio appropriato in base all'esito ricevuto.

7. Protocollo di comunicazione e strutture dati

Client e server comunicano scambiando oggetti Java serializzati (`ObjectOutputStream` / `ObjectInputStream`) su un socket TCP. Ogni scambio è composto da una `Richiesta` (un `Comando` più un array di parametri posizionali) e una `Risposta` (esito, messaggio, dato opzionale).

Le strutture dati scambiate sul filo sono i DTO del package `cinemax.common.dto`: `FilmDTO`, `ProiezioneDTO` (include il campo calcolato `postiLiberi`), `UtenteDTO` (non contiene mai la password né il suo hash) e `PrenotazioneDTO` (già arricchito con i dati di proiezione e cliente, per evitare richieste aggiuntive lato client).

Il contratto esatto dei parametri per ciascun Comando è documentato nel Javadoc della classe `Dispatcher`, generata insieme al resto della documentazione (vedi `doc/javadoc`).